Challenge Brief Summary — Signalpost / Builderr.ai

Source: Builderr.ai challenge page (condensed from the shared brief; always cross-check against the official downloadable brief before building — this file is a working summary, not the authoritative spec).

Basics

• Platform: Builderr.ai

• Challenge: Signalpost company research

• Window: Aug 23 – Oct 21, 2026

• Prize pool: $2,500 main + $250/$150/$100 JBOX bonus + $100 x 4 fortnightly vote

• Upside: possible partnership with Håvard Liltved Dalen (CPO at Fronted, JBOX co-founder) to launch Signalpost in Norway

What to build

An agent that discovers, matches, and refreshes verified company profiles for Norway-registered companies, built entirely from official filings and verifiable public sources.

Universe

• 411,160 companies — the frozen 2025-filer universe, full list public

• Builderr does **not** supply official sites or social identities — finding/proving those connections is the core challenge

Submission requirements

• Minimum 1,000 completed profiles in the public entry

• 10,000+ or the full universe welcomed if the system can handle it

• Submit: org-number manifest, one run command, models/APIs used, licences, expected cost per 100-company batch

Daily evaluation

• Same random 100 companies for every frozen agent, selected after daily cutoff

• Scored on evidence, freshness, repeatability

• Exactly 100 terminal result envelopes required per batch (hard gate)

Locked run budget (per 100-company batch)

Resource	Limit
Wall-clock	45 minutes
Compute	8 vCPU, 16 GB RAM, 10 GB temp disk
Outbound requests	2,000 max (cache hits free; redirects/retries count)
External API spend	$10 max declared (Builderr-supplied snapshots excluded)
Revisions	up to 4 new commit hashes before Oct 18, frozen before next run

Scoring (100 pts total, qualification bar 65/100)

Category	Weight	Notes
Coverage & source discovery	35	find real evidence, not just easy companies/registry record
Accuracy, identity & evidence	30	exact legal entity, supported claims, dates/periods preserved
Refresh & extensibility	20	reliable on random companies, revisits sources, preserves history
Decision-useful synthesis	10	helps someone understand company + what changed quickly
UX & interaction	5	find/compare/question/verify, desktop + mobile

Per external field: 70% of coverage score = company recall, 30% = claim recall.

Every verified discovery expands a versioned union that all entrants are rescored against.

Qualification requires ALL of:

• Overall score ≥ 65/100

• Coverage ≥ 21/35

• Weighted external company recall ≥ 60%

• External precision ≥ 95%

• Every hard gate below

Hard gates (any failure = disqualifying)

1. ≥21/35 coverage and ≥60% weighted external company recall
2. ≥95% external precision, no material wrong-company publication
3. Exactly 100 terminal result envelopes per daily batch
4. No fabricated financial values
5. Claim-level source, retrieval time, and reporting period where relevant
6. Missing / blocked / not-applicable stay distinct — never silently zero
7. Idempotent refresh with previous snapshot preserved
8. Documented source rights, server-side secrets, safe URL handling

Recommended stack (from the brief)

• Scrapy — crawl control
• extruct + Trafilatura — structured + text extraction
• Playwright — fallback only, not default
• Pydantic — validation
• RapidFuzz — candidate matching

Learning harness loop (their framing)

1. Try several routes (static HTML, structured data, sitemaps, targeted pages, search candidates, browser fallback) on the dev slice
2. Score every attempt: exact-company matches, supported fields, bad claims, runtime, requests, cost
3. Promote a new strategy only if it adds coverage without increasing wrong-company or unsupported-claim errors
4. Freeze routes/prompts/thresholds before the next daily cutoff — the random run tests the frozen system, it is not same-day training data

Materials provided by Builderr

• Full 411,160-company universe (compressed JSONL, published hashes)
• Runnable reference agent (Python baseline, tests, batch selector, crawler scripts, one-command runner)
• Challenge brief (universe, output contract, locked format, dates, scoring)
• Agent playbook (crawlers, fuzzy identity resolution, provenance, refresh)

• Learning harness guide

• Source policy (official/company-owned/licensed/prohibited boundaries)

• Evaluation contract (gates, metrics, budget, tie-breaks, corpus hashes)

• 100-company sample output (product shape reference)

Prior related experience (context, not part of the official brief)

Placed 3rd of 38 in Builderr.ai's earlier trading-agent challenge (Round 1, closed Jul 2) with a Python momentum/volatility strategy and a React + FastAPI + SQLite dashboard — same general toolchain fits here, swapping strategy logic for crawl/match logic.